Parte 1 · Calentamiento (10 min)

Escribe V o F en cada frase. Se hace solo y sin mirar las claves hasta acabar la parte.

Hora de inicio: Hora de fin:

1. start() ejecuta la tarea en el hilo main. __F___
2. Programación concurrente y programación paralela son lo mismo. __F___
3. Dos hilos del mismo proceso comparten memoria. __V___
4. getInputStream() de un Process es lo que el hijo recibe. __F___
5. waitFor() bloquea al padre hasta que el hijo termina. __V___
6. Crear un hilo es más costoso que crear un proceso. __F___
7. Tres hilos de 10, 4 y 5 segundos lanzados con start() tardan 19 segundos en total. __F___
8. Internet es un ejemplo típico de programación distribuida. __V___
9. Por convenio, un código de salida distinto de 0 indica error. __V___
10. Un mismo objeto Thread se puede arrancar dos veces con start(). __F___

Parte 2 · Teoría (30 min)

Responde cada pregunta con tus palabras, en unas 5 líneas y sin apuntes (7 minutos por pregunta).

Hora de inicio: 19:20 Hora de fin: 19:41

Pregunta 1

¿Qué diferencias hay entre un proceso y un hilo, y cómo se relacionan?

Un proceso es independiente y dentro tiene hilos, los hilos digamos que son pequeños brazos

que se dedican a pequeñas tareas del mismo proceso.

Cada proceso tiene su memoria asignada independiente, y cada hilo comparte la memoria

asignada a su proceso

Crear un hilo es más rápido que crear un proceso.

Si un hilo muere por un error puede matar el proceso que lo corre, sin embargo si un proceso

muere por algun error es independiente y no afecta a otros procesos

Pregunta 2

¿Qué es la programación concurrente y qué problemas plantea (exclusión mutua y condición de sincronización)? Pon un ejemplo de cada uno.

La programación concurrente es cuando un programa tiene varias tareas (hilos o procesos) funcionando al mismo tiempo, ya sea a la vez en varios núcleos o turnándose en uno solo. El problema es que muchas veces comparten cosas, como una variable o un fichero, y eso da dos problemas.

La exclusión mutua es que hay recursos que solo puede usar una tarea a la vez, porque si entran dos se estropean los datos. Por ejemplo, si dos hilos sacan 50 € de una cuenta con 100 € a la vez, los dos leen 100 y el saldo se queda en 50 en vez de 0. En Java se arregla con synchronized.

La condición de sincronización es cuando una tarea tiene que esperar a que otra haga algo antes de seguir. Por ejemplo, el padre que hace waitFor() y espera a que el hijo termine, o un consumidor que no puede coger datos hasta que el productor los ha metido.

Pregunta 3

¿Qué diferencias hay entre programación paralela y programación distribuida? Ventajas e inconvenientes de cada una.

La programación paralela es cuando un programa se reparte entre varios núcleos o procesadores del mismo ordenador, que trabajan a la vez y comparten la misma memoria. La programación distribuida es cuando el trabajo se reparte entre varios ordenadores distintos conectados por red, cada uno con su propia memoria, y se comunican mandándose mensajes. Un ejemplo típico es Internet.

La paralela tiene como ventaja que es muy rápida, porque los núcleos se comunican al momento al compartir memoria. Como inconveniente, está limitada por los núcleos que tenga el ordenador, el hardware potente es caro y hay que tener cuidado con la sincronización.

La distribuida tiene como ventaja que se puede ampliar todo lo que quieras añadiendo más ordenadores, y si uno falla los demás siguen funcionando. Como inconveniente, depende de la red, que puede ir lenta o caerse, es más difícil de programar y hay más problemas de seguridad.

Parte 3 · Laboratorio Java (35 min)

Para cada reto: 1) escribe tu predicción, 2) programa, 3) ejecuta y anota qué ha pasado, 4) si no coincide, explica por qué. Si te atascas, usa las pistas por orden. Los programas Eco y Espera están en material/.

Hora de inicio: Hora de fin:

Reto 1 · Leer la salida de un proceso (8 min)

Lanza java -version desde Java y muestra lo que imprime, junto con su código de salida.
Predice: ¿lo leerás con getInputStream()?

Mi predicción:
Creo que sí, que lo leeré con getInputStream(), porque es la salida del proceso hijo. El código de

salida será 0 porque el comando funciona bien.

Qué ha pasado al ejecutarlo:

Por getInputStream() no ha salido nada. Todo el texto de la versión openjdk version "25.0.1"etc ha salido por getErrorStream(). El código de salida ha sido 0. (osea que funciona)

Si no coincide, por qué:
No coincide porque java -version escribe la versión por la salida de errores y no por la salida

normal. Aunque no haya ningún error, ese texto va por getErrorStream(), y el código 0 confirma

que el comando ha ido bien.

Ampliación: lanza un comando que no exista y captura la excepción. ¿Qué mensaje da?
Lanza una IOException con el mensaje: Cannot run program "comandoquenoexiste":

CreateProcess error=2, El sistema no puede encontrar el archivo especificado. El proceso ni

siquiera llega a crearse, por eso no hay código de salida.

Pistas (solo si te atascas):

- Pista 1: new ProcessBuilder("java", "-version").start().
- Pista 2: mira también getErrorStream(); el código lo da waitFor().

Reto 2 · Enviar datos al hijo (10 min)

El programa Eco lee líneas de su entrada hasta recibir fin y las devuelve en mayúsculas. Haz un padre que le envíe hola, mundo y fin, y muestre lo que responde.

Predice: ¿qué pasa si no envías fin ni cierras el flujo?

Mi predicción:

Qué ha pasado al ejecutarlo:

Si no coincide, por qué:

Ampliación: espera con waitFor(3, TimeUnit.SECONDS) y, si no termina, destroy().

Pistas (solo si te atascas):

- Pista 1: getOutputStream() envuelto en un PrintWriter con autoflush.
- Pista 2: haz close() al terminar. Para lanzar Eco, usa "java", "-cp", System.getProperty("java.class.path"), "Eco".

Reto 3 · Secuencial frente a paralelo (10 min)

Espera duerme los segundos que le indiques. Lanza dos hijos de 2 s: primero uno tras otro, y luego los dos a la vez. Mide el tiempo de cada versión.

Predice: ¿cuánto tardará cada una?

Mi predicción:

Qué ha pasado al ejecutarlo:

Si no coincide, por qué:

Ampliación: ¿cuánto tardan 3 hijos de 2 s en paralelo?

Pistas (solo si te atascas):

- Pista 1: System.currentTimeMillis() antes y después.
- Pista 2: en paralelo, haz start() de los dos antes del primer waitFor().

Reto 4 · Hilos con Runnable y join() (7 min)

Crea 3 hilos que duerman 1, 2 y 3 segundos e impriman su nombre al acabar. El main debe imprimir cuánto ha tardado en total cuando terminen los tres.

Predice: ¿cuánto tarda? ¿Y si pones el join() justo después de cada start()?

Mi predicción:

Qué ha pasado al ejecutarlo:

Si no coincide, por qué:

Ampliación: quita los join(). ¿Qué imprime ahora el main y cuándo?

Pistas (solo si te atascas):

- Pista 1: una clase que implemente Runnable y reciba los segundos.
- Pista 2: guarda los hilos en un array y haz los join() en un segundo bucle.

Parte 4 · Simulacro (75 min)

Respeta el tiempo: cuando se acabe, para y guarda lo que tengas.

Teoría (25 min, sin internet): ua1ex1_teoria.txt

Hora de inicio: Hora de fin:

Respuestas:

Práctica (50 min): ua1ex1_practica.txt

Crea Hijo.java y Padre.java.

Hora de inicio: Hora de fin:

Parte 5 · Autoevaluación

Con las claves de claves/ rellena esta hoja.

V/F (10 frases)	____ /10	Qué fallé:
Teoría: proceso frente a hilo	____ /3	Qué fallé:
Teoría: concurrencia y problemas	____ /3	Qué fallé:
Teoría: paralela frente a distribuida	____ /3	Qué fallé:
Reto 1	OK / No	Qué fallé:
Reto 2	OK / No	Qué fallé:
Reto 3	OK / No	Qué fallé:
Reto 4	OK / No	Qué fallé:
Simulacro teoría	____ /10	Qué fallé:
Simulacro práctica	____ /10	Qué fallé:

Lo que voy a repasar antes del examen:

1.

2.

3.